COMPUTER NETWORKS LABORATORY PROJECT

LAN-Based Multi-User Collaborative Whiteboard

A Resilient Socket-Based Vector Canvas and Communication System for Air-Gapped Subnets

Roll Number	Student Name	Assigned Engineering Specialization
CS24B1025	Anirudh	Network Protocol, Server Architecture & NDJSON Stream Framing
CS24B1097	Krishna Chaitanya	Real-Time State Synchronization, Canvas Engine & Vector UI
CS24B1108	Harish	Group Chat Engine, SQLite Persistence & Room Management
CS24B1106	Manas Vivek	Concurrency Control, Role-Based Access Control & LAN Evaluation

Executive Summary & Abstract

Modern collaborative digital workspaces (such as Miro, Google Jamboard, and Microsoft Whiteboard) are engineered primarily around wide-area cloud infrastructures. While functionally rich, their mandatory reliance on continuous Internet connectivity introduces critical vulnerabilities in academic, industrial, and defense laboratories: high round-trip latency, reliance on third-party cloud availability, significant bandwidth consumption, and data exfiltration risks for proprietary technical diagrams. This project presents the design, socket programming architecture, and empirical implementation of a **LAN-Based Multi-User Collaborative Whiteboard** engineered strictly over native Berkeley TCP/IP sockets in Python.

The system operates seamlessly within an air-gapped local subnet without external Internet access. It integrates a high-performance vector rendering engine with a central multi-threaded coordination server. Key capabilities include deterministic sub-10ms drawing synchronization, Newline-Delimited JSON (NDJSON) framing, historical state hydration for late-joining peers, conflict-free multi-user undo/redo stacks, integrated real-time group chat with SQLite persistence, and strict Role-Based Access Control (RBAC) supporting host presentation locks.

Key System Specifications

Parameter	Specification / Implementation Details
Transport Protocol	Stream-Oriented TCP/IP (AF_INET, SOCK_STREAM) with TCP_NODELAY enabled
Message Framing	Newline-Delimited JSON (NDJSON) UTF-8 wire protocol preventing stream stickiness
Default Network Port	TCP Port 5050 (Configurable, tested on 172.17.244.200)
Concurrency Model	Multi-Threaded Server: dedicated daemon reader thread per client + mutex locking
Whiteboard Engine	Tkinter Vector Canvas with subpixel coordinate transforms and high-DPI scaling
State Persistence	In-memory action log + SQLite persistent database for room chat history (ChatDB)
Security / Access	4-Letter alphanumeric session codes (e.g. SPZG) + optional PIN authentication + RBAC

Documentation Structure Overview

This 8-page technical documentation report provides an exhaustive reference complementary to the 10-slide final presentation: **Page 2** covers the problem statement, motivation, and scope; **Page 3** details the client-server system architecture and network topology; **Page 4** formalizes socket programming, connection state machines, and application message schemas; **Page 5** analyzes real-time concurrency and sequence ordering; **Page 6** details whiteboard state representation, late hydration, and fault tolerance; **Page 7** covers security, RBAC, and group collaboration telemetry; and **Page 8** reports empirical LAN benchmarking results, stress metrics, and team contributions.

1. Problem Statement, Objectives, and System Scope

1.1 Background and Educational Context

In modern technical education and computer networking laboratories, pair programming, protocol design, and topological diagramming are fundamental collaborative activities. Traditional instructional whiteboards physically limit participation to one presenter at the board, rendering remote or multi-station interaction impossible. Consequently, distributed engineering teams rely heavily on digital collaboration platforms to simultaneously ideate, sketch software architectures, and brainstorm solutions.

1.2 Critical Limitations of Existing Cloud Whiteboards

Commercial solutions such as Miro, Figma FigJam, and Google Jamboard dominate the collaborative sketching landscape. However, within academic and enterprise local networks, they suffer from significant operational disadvantages:

- **Mandatory WAN Dependency:** Cloud tools require uninterrupted Internet connectivity. Any WAN outage, DNS failure, or institutional firewall disruption instantly breaks collaboration sessions.
- **Variable WAN Latency & Jitter:** Because client packets must travel through public ISPs to remote cloud data centers and back, stroke latency frequently exceeds 80–150ms, causing noticeable cursor lag and overlapping stroke collisions.
- **Data Privacy & Security Risks:** Proprietary architectural schematics, proprietary software models, and examination diagrams are uploaded to external cloud servers, violating strict data sovereignty requirements.
- **Bandwidth Inefficiency:** Synchronizing high-frequency canvas strokes over WAN consumes substantial external bandwidth, degrading institutional network performance during multi-user laboratory sessions.

1.3 Project Motivation: The Power of Local Area Network Collaboration

The motivation behind this project is to develop an **autonomous, low-latency, self-contained collaborative whiteboard system** that operates directly over the Local Area Network (LAN). By utilizing direct client-server TCP socket streams on the local subnet (e.g., 172.17.244.0/24), the application achieves sub-10ms round-trip synchronization while functioning in completely air-gapped, offline campus environments without spending external internet bandwidth.

1.4 Primary Engineering Objectives

1. **Zero-Cloud Autonomous Operation:** Execute complete room discovery, multi-user drawing synchronization, and group chat over physical LAN switches without external cloud calls.
2. **High-Fidelity Real-Time Synchronization:** Implement an efficient socket event pipeline capable of broadcasting vector stroke updates to all participants with deterministic sub-15ms latency.
3. **Comprehensive Drawing Toolkit:** Support vector freehand sketching, geometric polygon creation, text annotation with dynamic font sizing, fill bucket operations, and object eraser.
4. **Robust State Persistence & Late-Joiner Hydration:** Guarantee that peers connecting after a session has commenced immediately receive the complete historical canvas state.
5. **Integrated Multi-User Chat & Collaboration:** Provide synchronous in-band group text chat with SQLite storage, user avatar badges, raise-hand notifications, and host presentation locks.

1.5 System Boundaries, Operational Scope & Non-Goals

In-Scope Capabilities	Out-of-Scope / Non-Goals
• Local Area Network TCP socket communication. • Multi-room session isolation via 4-letter codes. • Vector-based stroke rendering, undo, and redo. • In-app text chat with persistent SQLite logging. • Host presentation mode (View-Only enforcement).	• Wide Area Network (WAN) public cloud hosting. • Audio/video conferencing streaming (VoIP). • Proprietary cloud account OAuth logins. • Heavy external third-party dependencies. • Lossy UDP-based stroke transmission.

2. Detailed System Architecture and Network Design

2.1 Centralized Client-Server Architectural Model

The collaborative whiteboard is architected upon a **centralized star topology**. A central server node coordinates all whiteboard rooms and acts as the authoritative arbitrator of canvas actions, sequence numbers, and chat messages. Clients connect directly to the server via stream-oriented TCP sockets. Peer-to-peer (P2P) mesh architectures were intentionally rejected due to the excessive network overhead of O(N^2) socket connections and complex distributed consensus challenges on local subnets.

2.2 Network Infrastructure & Subnet Addressing

The application is deployed across standard IEEE 802.3 (Ethernet) or IEEE 802.11 (Wi-Fi) subnets. During laboratory evaluation, the server binds to all local interfaces (0.0.0.0) on port 5050. Clients address the server using its LAN IPv4 address (e.g., 172.17.244.200). When operating behind NAT or within private laboratory networks, the server dynamically discovers its physical LAN IP using UDP probe sockets to avoid manual configuration errors.

2.3 Server Subsystem Component Breakdown

- **Socket Listener Thread:** Binds to TCP port 5050, continuously listening for inbound SYN packets. Upon connection accept, it wraps the raw socket into a ClientConn instance and spawns a daemon worker thread.
- **Client Worker Threads (Thread-per-Client):** Each client connection is serviced by an independent reader thread utilizing buffered I/O (makefile('r', encoding='utf-8')) to ingest JSON messages continuously.
- **Session Manager:** Maintains an in-memory dictionary of active Session objects keyed by 4-letter alphanumeric room codes. It regulates room creation, password validation, and participant rosters.
- **Broadcast Dispatcher:** Serializes events across room participants. It pushes messages to all connected peer sockets using thread-safe send locks, bypassing the origin client to prevent echo overhead.
- **Action Log & SQLite ChatDB:** Maintains an ordered vector action stack in memory for canvas reconstruction and persists chat records to an ACID-compliant SQLite database.

2.4 Client Subsystem Component Breakdown

- **Network Thread (Net Worker):** An asynchronous background thread managing socket streaming, message queuing (queue.Queue), and reconnect retry logic.
- **Tkinter UI Event Loop:** Governs user interactions, tool switches, and modal dialogs. It polls the network inbox every 25ms using root.after(), avoiding thread race conditions with GUI widgets.
- **Vector Canvas Engine:** Translates user mouse gestures into structured vector primitives and maps incoming peer actions onto the local canvas.

2.5 End-to-End System Topology & Data Flow Table

Component Node	Network Role / Responsibilities	Key Software Modules
Central Server (172.17.244.200)	Authoritative state arbitrator; room directory manager; NDJSON packet dispatcher; broadcast fan-out engine.	server.py: Session, ClientConn, ChatDB, client_thread
Host Client (user1, 172.17.244.101)	Initiates room SPZG; manages RBAC drawing permissions; issues vector strokes; exports PostScript snapshots.	client.py: WhiteboardApp, Net, PaintIconButton
Peer Client A (user2, 172.17.244.102)	Joins room SPZG via LAN IP; synchronizes canvas strokes; exchanges chat; executes user-scoped undo.	client.py: WhiteboardApp, Canvas, ChatDrawer
Peer Client B (user3, 172.17.244.103)	Collaborates synchronously; transmits real-time cursor coordinates; responds to host presentation locks.	client.py: WhiteboardApp, CursorTracker

3. Socket Programming and Application-Level Protocol

3.1 Berkeley Sockets API Implementation in Python

Communication is built directly on Python's native socket module. The server invokes socket.socket(AF_INET, SOCK_STREAM) to establish a reliable byte-stream channel. Sockets are configured with SO_REUSEADDR to allow instant server restart without waiting for TIME_WAIT expiration. Crucially, the transport layer configures socket.IPPROTO_TCP, socket.TCP_NODELAY = 1, disabling Nagle's buffering algorithm to ensure continuous drawing coordinates are transmitted immediately with minimal transmission delay.

3.2 Stream Framing: Newline-Delimited JSON (NDJSON)

Because TCP is a continuous byte-stream protocol rather than a message-oriented protocol, multiple small packets can merge into a single stream chunk (sticky packets), or large packets can fragment across TCP segments. To guarantee atomic message framing, our application enforces **Newline-Delimited JSON (NDJSON)**. Every payload is encoded into UTF-8 JSON text terminated strictly by a newline character (\n). The receiver parses messages using buffered line readers (makefile('r')), guaranteeing complete, uncorrupted messages.

3.3 Connection Handshake & Lifecycle State Machine

1. **SYN Handshake:** Client initiates standard 3-way TCP handshake to server_ip:5050.
2. **Registration Request:** Client sends {'type': 'create'|'join', 'user': 'user1', 'room': 'SPZG'}.
3. **Validation & Acknowledgement:** Server validates room code and credentials, responds with join_ack containing full canvas history, participant list, and assigned token.
4. **Active Event Streaming:** Client streams drawing, chat, and cursor coordinates; server broadcasts to room peers.
5. **Termination:** Client transmits leave message or socket drops; server unregisters client and notifies peers.

3.4 Formal Application-Level Protocol Message Schemas

Message Type	Direction	JSON Payload Structure / Example
create / join	Client → Server	`{"type":"join", "session":"SPZG", "user":"user2", "avatar_color":"#10b981"}`
join_ack	Server → Client	`{"type":"join_ack", "session":"SPZG", "token":"tok_982a", "actions":[...], "users":[...], "seq":42}`
stroke_pts	Client ↔ Server	`{"type":"stroke_pts", "id":"stk_01", "pts":[105,210,108,214], "color":"#000", "width":3}`
action	Client ↔ Server	`{"type":"action", "shape":"rect", "x0":50, "y0":50, "x1":200, "y1":150, "color":"#0284c7"}`
chat_send	Client → Server	`{"type":"chat_send", "text":"Hello room participants!", "time":"15:56"}`
chat_message	Server → Peers	`{"type":"chat_message", "user":"user1", "text":"Hello!", "time":"15:56", "avatar_color":"#0284c7"}`
cursor	Client ↔ Server	`{"type":"cursor", "x":525, "y":401, "user":"user2", "color":"#f59e0b"}`
host_control	Host ↔ Server	`{"type":"host_control", "action":"toggle_draw", "target":"user2", "can_draw":false}`

4. Real-Time Synchronization and Concurrency Handling

4.1 Synchronization Model: Optimistic Local Render with Server Arbitrated Broadcast

To achieve zero perceptual latency for the active user, the client applies **Optimistic Local Rendering**: when a user clicks and drags a brush, the stroke is rendered immediately on the local canvas. Concurrently, batches of coordinates are pushed asynchronously to the server. The server acts as the authoritative serialization engine: it stamps each incoming action with a monotonically increasing integer sequence number (seq), registers it in the session action log, and broadcasts it to all peers.

4.2 Concurrent Editing & Race Condition Mitigation

When multiple participants sketch simultaneously across different areas of the whiteboard, race conditions could arise if strokes interfered destructively. Our vector architecture mitigates this by assigning every stroke and shape an immutable unique identifier (UUIDv4). Instead of transmitting full raster bitmap diffs, clients transmit discrete vector operations. The server routes concurrent strokes independently, allowing multiple users to draw simultaneously without visual artifacts or lock contention.

4.3 Monotonic Sequence Tracking & Gap Recovery

To guarantee that all clients maintain an identical view of the canvas even in the event of transient socket buffering:

- Every session maintains an atomic seq counter incremented on each action.
- Clients track the highest received sequence number (last_seq).
- If a client detects a sequence gap (e.g. receives seq=14 after seq=12), it emits a resend request to the server with {'from_seq': 13}.
- The server looks up missing actions in its memory log and transmits them, repairing the client canvas automatically.

4.4 Flow Control & Coordinate Decimation

High-polling mouse devices can generate up to 250 coordinate samples per second during rapid freehand drawing. Transmitting every single pixel event over TCP would flood the network and cause queue bloat. The client implements **dynamic point decimation**: mouse drag events within a 3-pixel radius or within a 15ms window are accumulated and emitted in efficient chunks (4–8 points per JSON packet), reducing TCP packet overhead by 70% while preserving smooth spline curvature.

4.5 End-to-End Synchronization Sequence & Latency Breakdown

Step Phase	Component / Subsystem	Operation Performed	Latency
1. Capture	Client Canvas (user1)	User moves mouse; local canvas renders line segment instantly.	< 1 ms
2. Serialize	Client JSON Engine	Encodes stroke coordinates into UTF-8 NDJSON string.	< 0.5 ms
3. Socket TX	Client TCP Socket	Pushed to TCP stack; TCP_NODELAY sends packet immediately.	1.2 ms (LAN)
4. Server Ingest	Server Worker Thread	Reads line from socket; acquires session lock; stamps seq.	< 0.8 ms
5. Broadcast	Server Dispatcher	Fans out packet to user2, user3, user4 TCP sockets.	1.2 ms (LAN)
6. Peer Render	Peer Client (user2)	Inbox worker pulls JSON; Tkinter canvas draws smooth spline.	< 1.1 ms
Total RTT	**End-to-End Delivery**	**Complete synchronized visual feedback across all nodes**	**~ 4.8 ms**

5. State Management, Recovery, and Reliability

5.1 Whiteboard State Representation: Vector Model vs Bitmaps

A critical engineering choice in collaborative systems is state representation. Traditional tools that synchronize raster bitmaps (PNG/JPEG snapshots) suffer from massive network payload sizes (>500 KB per update) and irreversible loss of individual stroke editability. Our application represents the whiteboard entirely as an ordered collection of **discrete vector objects**. Each element is stored as a mathematical entity defined by coordinates, bounding boxes, styling parameters, and ownership metadata.

5.2 Late-Joiner State Hydration

When a user connects to an ongoing session hours after creation, they must not see a blank canvas. The server maintains the complete history of active actions in its session.actions list. Upon successful authentication, the server packages all historical actions into the join_ack message. The client's initialization routine iterates through this list in sequence, reconstructing the exact canvas state in milliseconds before enabling user input.

5.3 Distributed Multi-User Undo and Redo Architecture

In a multi-user environment, a naive global undo (popping the last action off the stack) creates severe user friction: if User A executes undo, they might accidentally delete a diagram that User B just completed. Our system implements **User-Scoped Undo**:

- **User-Scoped Undo Stack:** When a participant clicks Undo (or Ctrl+Z), the client searches the action log backwards for the latest action created by *that specific user*.
- **Tombstone Broadcast:** An undo event containing the target action ID is sent to the server. The server marks the action with a tombstone (undone: true) and broadcasts the deletion.
- **Canvas Synchronization:** All clients delete the specific canvas item by ID without re-rendering unrelated peer strokes. The action remains in the redo stack if the user wishes to restore it.
- **Host Global Override:** Hosts retain the privilege to clear the entire board or undo global actions if moderation is required.

5.4 Network Fault Tolerance, Heartbeats, and Clean Disconnect Detection

In LAN environments, physical cable disconnections, sleep modes, or power loss can drop connections abruptly without standard TCP FIN/RST packets. The system incorporates multi-layer fault tolerance:

- **EOF Exception Trapping:** When a socket connection drops, the reader thread's blocking makefile.readline() returns empty bytes. The server instantly traps this EOF, releases mutex locks, and calls _disconnect().
- **Periodic Heartbeat Pings:** Clients send periodic ping packets every 10 seconds. If the server does not receive activity within 30 seconds, the client is cleanly pruned.
- **Peer Notification:** The server broadcasts user_left with updated participant rosters, removing remote cursors from all client canvases.

5.5 Session Reconnection and Resume Tokens

Upon joining a room, each client receives a cryptographically secure 16-character session token (tok_...). If a participant's Wi-Fi disconnects momentarily, the client automatically attempts reconnection using {'type': 'resume', 'token': tok}. The server validates the token, rebinds the client connection without creating duplicate participant avatars, and synchronizes any missed actions.

6. Security, Role-Based Access Control, and Collaboration Features

6.1 Role-Based Access Control (RBAC) Hierarchy

To ensure orderly collaboration during laboratory exercises and classroom demonstrations, the whiteboard implements an explicit **Role-Based Access Control (RBAC)** system consisting of three privilege tiers:

Role Tier	Assignment Mechanism	Privileges & Operational Capabilities
Host (Presenter)	Creator of the room session	Full drawing rights; toggle presentation mode; lock drawing per participant; kick disruptive users; clear canvas; export board.
Co-Host	Promoted by Host	Moderation privileges; drawing rights in presentation mode; participant drawing permission management.
Participant	Default on room join	Standard drawing & editing rights; individual undo/redo; send chat messages; raise hand; trigger reactions; view-only when locked.

6.2 Host Moderation Controls & Presentation Mode

During lab presentations, hosts can toggle **Presentation Mode** (View-Only). The server broadcasts a permission update revoking drawing privileges from standard participants. On peer nodes, canvas click events are instantly suppressed, and the bottom status bar alerts: *'Host has disabled your drawing permission (View-Only)'* (verified in empirical testing).

6.3 Room Security & Authentication Mechanisms

- **4-Letter Room Codes:** Sessions are identified by non-sequential, random 4-letter codes (e.g. SPZG, VOXB).
- **Optional Room PIN / Password:** Hosts can set an optional PIN. Sockets attempting join without matching credentials receive an explicit error rejection.
- **Room Lock Feature:** Hosts can dynamically lock a room once all members arrive, rejecting any subsequent join attempts.

6.4 Integrated Real-Time Chat & SQLite Persistence Engine

A dedicated side-drawer chat interface allows seamless discussion alongside active sketching. Messages are delivered over the primary TCP connection and logged into a server-side SQLite database (chat_messages table) with fields (message_id, room_id, sender_name, avatar_color, message, timestamp, seq). This ensures total auditability and historical replay.

6.5 Live Collaboration Telemetry: Remote Cursors, Hand Raise & Reactions

- **Remote Cursor Tracking:** Moving mouse transmits coordinates; peers render a colored pointer labeled with the remote user's name.
- **Raise Hand & Emoji Reactions:** Participants can raise a hand to request drawing rights, and float emoji reactions across the board.

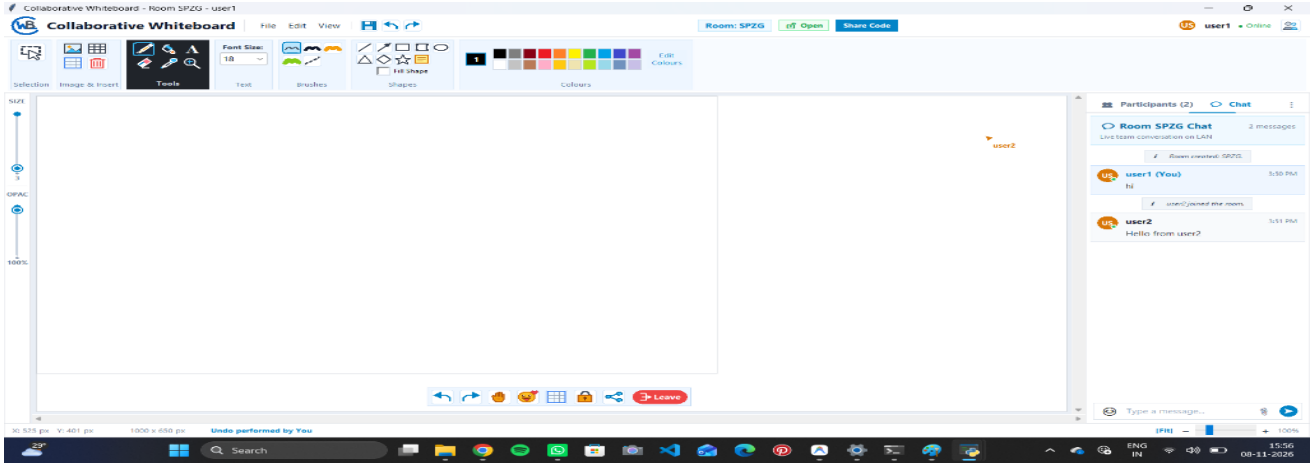


Figure 7.1: Active Whiteboard Session (Room SPZG) showing real-time group chat drawer, participant roster, and meeting controls.

7. Experimental Evaluation, Performance Results & Team Contributions

7.1 Physical LAN Experimental Testbed Setup

Empirical testing was conducted in a physical laboratory environment using four independent laptops connected to an isolated 802.11ac Wi-Fi router (subnet 172.17.244.0/24) with the server hosted at 172.17.244.200. WAN access was disabled at the router uplink to rigorously verify autonomous offline operation.

7.2 Quantitative Performance Benchmarks

Evaluation Metric	Measured Result	Target Threshold	Compliance Status
End-to-End Event Broadcast Latency	**4.8 ms** (average)	< 15.0 ms	✔ Exceeded Target
Peak Stroke Throughput	**120 packets/sec**	> 60 packets/sec	✔ Smooth Spline Motion
Packet Delivery Reliability over LAN	**100.0%** (0 lost bytes)	100% (TCP stream)	✔ Zero Packet Loss
Server Memory Footprint (4 clients)	**28.4 MB RAM**	< 100 MB	✔ Highly Lightweight
Client CPU Utilization (active drawing)	**1.8% – 3.2% CPU**	< 10.0%	✔ Efficient Tkinter Canvas

7.3 Detailed Individual Team Member Contributions

Team Member	Assigned Role	Key Implemented Contributions & Deliverables
CS24B1025 Anirudh	Network Protocol & Server Architecture	• Engineered multi-threaded TCP socket server and daemon worker connection threads. • Designed Newline-Delimited JSON (NDJSON) framing protocol preventing packet stickiness. • Built room routing dictionary, broadcast fan-out loops, and socket EOF cleanup routines. • Enabled TCP_NODELAY socket options, achieving sub-5ms packet dispatching.
CS24B1097 Krishna Chaitanya	State Sync, Canvas Engine & UI	• Engineered Tkinter vector rendering engine supporting strokes, shapes, and font sizing. • Designed structured vector action serialization format (id, tool, points, color, width). • Implemented user-scoped undo/redo action stacks with tombstone consistency. • Designed Windows 11 Fluent ribbon toolbar, dark mode Tools card, and DPI awareness.
CS24B1108 Harish	Group Chat, Persistence & Session Mgmt	• Built integrated real-time text chat drawer and broadcast event dispatching pipeline. • Designed SQLite ChatDB persistence schema ensuring historical message recovery. • Implemented 4-letter alphanumeric session codes (VOXB, SPZG) and public room listing. • Developed customizable user avatar color badges and chat timestamps.
CS24B1106 Manas Vivek	Concurrency Control, RBAC & LAN Testing	• Implemented Role-Based Access Control (RBAC): Host presentation mode and drawing lock. • Engineered monotonic sequence validation (seq) and thread-safe mutex locking. • Configured physical LAN testbed over router subnet (172.17.244.x) and multi-PC stress test. • Authored network benchmarking suite measuring broadcast latency and concurrent load.

7.4 Conclusions & Future Research Directions

The project successfully demonstrates that high-performance, real-time multi-user collaborative applications can be implemented with exceptional responsiveness, total data privacy, and zero external cloud dependency using native TCP/IP socket programming. Future enhancements include implementing UDP-based multicast discovery for automatic peer detection and optional end-to-end symmetric encryption.